Here are **detailed Week 9 notes** for **Neural Networks — Session XVI**.

Week 9 — Neural Networks

Machine Learning and Advanced Analytics: Session XVI

1. Machine learning as function learning

A useful way to understand machine learning is:

> *Machine learning is about learning a function that maps inputs to outputs.*

For regression:

```
predicted_sales = f(price, promotion, store_size, day_of_week)
```

For classification:

```
churn_probability = f(recency, frequency, spend, complaints)
```

The Neural Networks I slides state this directly: machine learning learns a mapping from inputs to an output, and different ML models approximate different types of functions. Linear predictors approximate linear functions, while random forests can approximate more arbitrary functions.

Part A — Artificial Neurons

2. What is an artificial neuron?

An artificial neuron is a very small function.

It:

```
1. Takes input values
2. Multiplies each input by a weight
3. Adds a bias
4. Applies an activation function
5. Produces an output
```

Basic structure:

```
output = activation(weighted_sum + bias)
```

or:

```
r = φ(w1x1 + w2x2 + ... + wnxn + b)
```

The slides describe a neuron as taking inputs, having one weight per input, having a bias, having an activation function, and computing an output.

---

3. Weighted sum

The first part of a neuron is the weighted sum.

Example:

```
age = 32
salary = 35,000

weight_age = 0.5
weight_salary = 0.001

weighted_sum = 32(0.5) + 35,000(0.001)
weighted_sum = 16 + 35
weighted_sum = 51
```

This is before applying the activation function.

Weights decide how strongly each input contributes to the neuron's output.

Bias shifts the function up or down, or moves the threshold/location of the activation.

---

4. Activation function

The activation function transforms the weighted sum.

Examples:

```
linear
step
sigmoid
ReLU
ELU
softmax
```

The activation function is important because it lets neural networks learn non-linear functions.

Without non-linear activation functions, stacking layers would still only produce a linear function.

Exam point

A linear function plus another linear function is still linear. Therefore, neural networks need non-linear activation functions to approximate complex functions.

---

5. A neuron is a generic basic function

A single neuron is very limited. It can represent a simple transformation of the inputs.

For example:

```
linear activation → behaves like linear regression
step activation → behaves like a threshold rule
sigmoid activation → outputs a smooth value between 0 and 1
```

So an individual neuron computes a basic generic function, but many neurons combined can approximate much more complex functions.

The slides summarise this as: neurons/perceptrons are very basic “generic” functions.

Part B — Shallow Neural Networks

6. What is a shallow neural network?

A **shallow neural network** usually means a network with:

```
input layer
one hidden layer
output layer
```

Example structure:

```
x1, x2, x3 → hidden neurons → output
```

The hidden layer contains multiple neurons. Each hidden neuron learns a simple function. The output layer combines these functions to produce the final prediction.

---

7. Why one neuron is not enough

One neuron can only produce a simple transformed weighted sum.

But most real-world prediction problems are not that simple.

For example, customer behaviour may depend on complex combinations of:

```
recent spending
long-term frequency
```

```
seasonality
discount sensitivity
complaint history
```

A single neuron cannot easily capture these relationships.

A shallow neural network solves this by combining many neurons.

---

8. Why shallow networks can approximate any function

The key idea is:

> *A hidden layer can create many simple shapes, and the output layer can combine them to approximate a complex function.*

The lecture uses a visual idea:

```
One pair/group of neurons can create a “bump”.
Many bumps can be added together.
Enough bumps can approximate any function.
```

This is the intuition behind the **universal approximation theorem**.

The slides explain that a shallow neural network with one hidden layer, a non-linear activation function, and enough neurons can approximate any function.

---

9. Bump intuition

Imagine a complicated curve.

Instead of trying to learn the whole curve directly, the network builds many small pieces:

```
bump 1 covers one region
bump 2 covers another region
bump 3 covers another region
...
```

The output layer adds these together.

With enough neurons, the network can create a very close approximation of the target function.

Important caveat

This does **not** mean a neural network will automatically learn the best function.

It only means the network has the capacity to represent the function if the correct weights and biases can be found.

---

10. Too many neurons

More neurons increase model complexity.

This can help if the model is underfitting, but it can also cause overfitting.

```
Too few neurons → underfitting
Too many neurons → overfitting and harder training
```

The slides compare this to increasing the depth of a decision tree: more complexity can fit the data better, but may generalise worse.

Part C — Training Neural Networks

11. What is training?

Training means learning the best values for:

```
weights
biases
```

The aim is to reduce prediction error.

The model starts with initial/random weights, makes predictions, calculates error, and updates weights to reduce that error.

---

12. Cost function

A **cost function** measures how wrong the model is.

For regression, common cost functions include:

```
MSE
MAE
```

For classification, common cost functions include:

```
cross-entropy
binary cross-entropy
categorical cross-entropy
```

Important distinction:

Concept	Meaning
Output function	The model's prediction function
Cost function	The error function we minimise

The Neural Networks I slides warn not to confuse the output function with the cost function. The output function maps inputs to predictions, while the cost function maps model parameters to error.

Part D — Gradient Descent

13. What is gradient descent?

Gradient descent is an optimisation algorithm used to minimise the cost function.

Core idea:

```
Start with random parameters.
Calculate the slope of the cost function.
Move parameters in the direction that reduces cost.
Repeat until improvement becomes small.
```

The slides describe gradient descent as tweaking parameters iteratively to minimise a cost function.

---

14. Hill analogy

Imagine standing on a hill in fog.

You cannot see the whole landscape, but you can feel the slope under your feet.

You move downhill step by step.

```
position = current weights and biases
height = cost/error
slope = gradient
step size = learning rate
```

The aim is to reach a low point where error is small.

---

15. Gradient

The **gradient** tells us:

```
which direction increases or decreases error
how steep the change is
```

For each parameter, gradient descent asks:

```
If I slightly change this weight, how does the cost change?
```

This is the partial derivative of the cost function with respect to that parameter.

---

16. Learning rate

The **learning rate** controls how big each update step is.

```
new_weight = old_weight − learning_rate × gradient
```

If learning rate is too small

```
Training is very slow.
The model may take too long to converge.
```

If learning rate is too large

```
The model may jump over the minimum.
Training may diverge.
The cost may increase instead of decrease.
```

The slides explain that a step size that is too low takes too long to converge, while a step size that is too high can diverge away from the minimum.

---

17. Local minima

Some cost functions are not smooth simple bowls. They may have many valleys.

A **local minimum** is a point that is lower than nearby points but not the best possible point overall.

Gradient descent can get stuck in local minima because it only follows local slope information.

---

18. Different feature scales and gradient descent

Gradient descent can struggle when variables have very different scales.

Example:

```
age: 18 to 80
salary: 15,000 to 100,000
```

Because the parameters interact with differently scaled variables, the cost surface can become stretched. Gradient descent may zig-zag and converge slowly.

The slides explain that slow convergence can be caused by variables on different scales, because gradient descent uses the same learning rate across directions.

Practical solution

Standardise or normalise input features.

Common approach:

```
mean = 0
standard deviation = 1
```

This makes gradient descent easier and faster.

Part E — Batch, Stochastic, and Mini-Batch Gradient Descent

19. Batch gradient descent

Batch gradient descent uses the full training dataset to calculate the gradient at every step.

```
Gradient calculated using all training examples.
```

Benefits

```
Stable gradient estimate
Usually smooth movement towards minimum
```

Drawbacks

```
Slow for large datasets
High memory cost
Each update is expensive
```

The slides state that batch gradient descent uses every training point at each step because the training points define the cost function.

---

20. Stochastic gradient descent

Stochastic gradient descent, or **SGD**, uses one training example at a time.

```
Gradient calculated using one random training example.
```

Benefits

```
Fast updates
Low memory cost
Can escape some local minima because of randomness
```

Drawbacks

```
Noisy updates
Cost may not decrease every iteration
Does not fully use parallelisation
May never settle exactly at the minimum
```

The slides explain that each SGD iteration is not guaranteed to reduce cost, but over time it moves close to the minimum on average.

---

21. Mini-batch gradient descent

Mini-batch gradient descent uses a small random subset of training examples.

```
Gradient calculated using a batch of, for example, 32, 64, or 128 rows.
```

Benefits

```
Less noisy than SGD
Faster than full batch
Can use parallel hardware efficiently
Common default in neural networks
```

Mini-batch gradient descent is between batch and stochastic gradient descent. The slides describe it as randomly selecting a subset, being less jumpy than SGD, and being able to take advantage of parallelisation.

---

22. Comparison table

Method	Data used per update	Pros	Cons
Batch GD	All rows	Stable updates	Slow and memory-heavy
Stochastic GD	1 row	Fast, may escape local minima	Very noisy
Mini-batch GD	Small subset	Good balance, parallelisable	Batch size must be chosen

---

Part F — Computing Gradients in Neural Networks

23. Forward pass

The forward pass means computing the prediction.

Steps:

```
1. Inputs enter the network.
2. Each neuron computes weighted sum + activation.
3. Outputs pass through layers.
4. Final prediction is produced.
5. Cost is calculated.
```

---

24. Backward pass / backpropagation

Backpropagation computes how much each weight contributed to the final error.

It works backwards:

```
output error
→ output layer weights
→ hidden layer weights
→ earlier hidden layer weights
```

The important intuition is:

> *The slope for each weight is computed backwards as a series of multiplications.*

The Neural Networks II slides describe the gradient computation as being computed backward through the network, and in deeper networks the middle part of this chain is repeated for lower layers.

---

25. Why backpropagation uses multiplication chains

Each weight affects the next neuron, which affects the next layer, which affects the final output, which affects the cost.

So the total effect is a chain:

```
change in weight
→ change in hidden neuron output
→ change in next layer output
→ change in prediction
→ change in cost
```

This is why gradients are calculated by repeatedly multiplying local slopes.

---

26. Why this matters

Because gradients are multiplied through layers, deep neural networks can suffer from:

```
vanishing gradients
exploding gradients
```

Although Week 9 focuses mostly on shallow networks, this intuition prepares for deep networks.

The Neural Networks II slides explain that because gradients are computed as a series of multiplications, the algorithm can get stuck. If values become too small, lower-layer weights are not updated; if values become too large, training can diverge.

Part G — Shallow Neural Network Gradient Intuition

27. Simple intuition

Suppose a hidden neuron feeds into an output neuron.

If we change one hidden-layer weight, the change affects:

```
hidden neuron output
then output neuron result
then cost/error
```

So the gradient asks:

```
How does changing this weight change the final cost?
```

This depends on:

```
the activation function slope
the next layer's weight
the current prediction error
```

---

28. Why activation function slope matters

If the activation function is flat at a point, then a change in input barely changes the output.

That means:

```
small activation slope → small gradient → slow learning
```

This is one reason activation function choice matters.

For example, a saturated sigmoid can produce very small gradients, making learning slow.

Part H — Neural Network Capacity and Overfitting

29. Capacity

Capacity means how complex a function the network can represent.

Capacity increases with:

```
more neurons
more layers
more parameters
more flexible activation functions
```

A shallow neural network can approximate any function with enough neurons, but using too many neurons can overfit.

---

30. Overfitting in neural networks

Overfitting happens when the network learns the training data too closely, including noise.

Signs:

```
training loss keeps decreasing
validation loss stops improving or increases
```

This means the model is becoming better on training data but worse on unseen data.

Part I — Deep Networks Context

31. Why use deep networks if shallow networks can approximate any function?

A shallow network can approximate any function theoretically, but it may require a very large number of neurons.

Deep networks can often represent useful functions more efficiently.

The Neural Networks II slides explain that deeper networks often perform better empirically, and a shallow network may need many more neurons, sometimes exponentially more.

---

32. Layers as feature learning

Each layer can be thought of as learning new features.

Example in image processing:

```
early layers: edges, lines, circles
later layers: eyes, faces, objects
```

Example in time series:

```
early layers: simple purchase/spend patterns
later layers: ratios or complex behavioural patterns
```

The slides state that each layer can be thought of as learning new features, such as lines/circles in image processing and ratios in time series.

Part J — Exam-Style Explanations

Q1. Explain machine learning as function learning.

Machine learning can be understood as learning a function that maps input variables to an output. In regression, the output is a continuous value, such as demand or sales. In classification, the output is a class or probability, such as churn/no churn. Different models approximate different kinds of functions; for example, linear regression approximates a linear function, while neural networks can approximate complex non-linear functions.

Q2. What does an artificial neuron do?

An artificial neuron takes input values, multiplies each input by a weight, adds a bias, applies an activation function, and outputs a value. The weights control the influence of each input, the bias shifts the function, and the activation function transforms the weighted sum, often introducing non-linearity.

Q3. What is a shallow neural network?

A shallow neural network is a neural network with one hidden layer between the input layer and output layer. The hidden neurons compute simple transformed weighted sums, and the output layer combines them to produce the final prediction. With enough neurons and a non-linear activation function, a shallow neural network can approximate any function.

Q4. Why can a shallow neural network approximate any function?

A shallow neural network can approximate any function because hidden neurons can form simple shapes such as steps or bumps, and the output layer can add these shapes together. With enough neurons, these small pieces can approximate an arbitrary curve or decision boundary. This is the intuition behind the universal approximation property.

Q5. What is gradient descent?

Gradient descent is an iterative optimisation method used to minimise a cost function. It starts with initial parameter values, calculates the slope of the cost function with respect to each parameter, and updates the parameters in the direction that reduces error. This is repeated until the model stops improving or reaches a stopping condition.

Q6. What is the learning rate?

The learning rate controls the size of each gradient descent update. If it is too low, training is slow and may take many iterations. If it is too high, the model may overshoot the minimum or diverge. Therefore, the learning rate must be chosen carefully.

Q7. Why do variables on different scales cause problems for gradient descent?

Variables on different scales stretch the cost surface, causing gradient descent to move inefficiently. The algorithm may take steps that are too large in one direction and too small in another, leading to zig-zagging, slow convergence, or divergence. Standardising features helps reduce this problem.

Q8. Compare batch, stochastic, and mini-batch gradient descent.

Batch gradient descent uses all training data for each update, giving stable but expensive updates. Stochastic gradient descent uses one training example per update, making it fast but

noisy. Mini-batch gradient descent uses a small subset of data, balancing speed, stability, and parallelisation. It is commonly used in neural network training.

Q9. What is the intuition behind computing gradients in a shallow neural network?

The gradient measures how changing a weight changes the final cost. In a neural network, changing a hidden-layer weight changes the hidden neuron output, which changes the final prediction, which changes the cost. Backpropagation computes this effect backwards using a chain of local slopes.

---

33. One-page revision summary

```
Week 9 focuses on neural networks as function learners.

Machine learning = learning a function from inputs to outputs.

Artificial neuron:
weighted sum + bias + activation function = output.

Weights:
control input importance.

Bias:
shifts the function.

Activation function:
transforms the weighted sum and introduces non-linearity.

Single neuron:
basic generic function.

Shallow neural network:
one hidden layer + output layer.

Universal approximation intuition:
hidden neurons create simple bumps/steps; many bumps can approximate any
function.

Training:
learn weights and biases by minimising a cost function.

Gradient descent:
iteratively updates parameters in the direction that reduces cost.

Learning rate:
controls update size.
Too low = slow.
Too high = overshoot/diverge.
```

```
Different feature scales:
make gradient descent slow or unstable, so standardisation helps.

Batch GD:
uses all data, stable but slow.

Stochastic GD:
uses one row, fast but noisy.

Mini-batch GD:
uses a subset, common practical compromise.

Backpropagation:
computes gradients backward through the network using chains of slopes.
```